

Manual Python Coding Interview Problems

Simple explanations and solutions without shortcut built-in functions

Prepared for Muhammad Shafiullah Khan

Note: These solutions avoid shortcut functions and methods such as `sort()`, `sorted()`, `reverse()`, `reversed()`, `set()`, `sum()`, `count()`, `index()`, `max()`, and `min()`. Basic language features such as loops, comparisons, indexing, and list append are used so the code remains practical Python.

1. Reverse a String

Simple idea

Read the string from the last character to the first character and build a new string.

How to solve it

1. Start from the final index.
2. Add each character to a new string.
3. Move backward until index 0.

Python code

```
def reverse_string(text):
    length = 0
    for character in text:
        length = length + 1

    result = ""
    index = length - 1

    while index >= 0:
        result = result + text[index]
        index = index - 1

    return result

print(reverse_string("Python")) # nohtyP
```

Example

Input: Python Output: nohtyP

Complexity

Time: $O(n)$. Space: $O(n)$.

2. Check Whether a String Is a Palindrome

Simple idea

A palindrome looks the same from both sides. Compare the first character with the last, then move both pointers inward.

How to solve it

4. Set one pointer at the beginning.
5. Set another pointer at the end.
6. If any pair is different, return False.
7. If all pairs match, return True.

Python code

```
def is_palindrome(text):
    length = 0
    for character in text:
        length = length + 1

    left = 0
    right = length - 1
```

```

while left < right:
    if text[left] != text[right]:
        return False
    left = left + 1
    right = right - 1

return True

```

```

print(is_palindrome("madam")) # True
print(is_palindrome("python")) # False

```

Example

madam is the same from left to right and right to left.

Complexity

Time: $O(n)$. Space: $O(1)$.

3. Find Duplicate Elements in a List

Simple idea

Compare every number with the numbers after it. Before adding a duplicate, check that it has not already been added.

How to solve it

8. Use an outer loop for the current number.
9. Use an inner loop to search later positions.
10. When equal values are found, check the result list.
11. Add the value only once.

Python code

```

def find_duplicates(numbers):
    duplicates = []
    i = 0

    while i < len(numbers):
        j = i + 1
        found_duplicate = False

        while j < len(numbers):
            if numbers[i] == numbers[j]:
                found_duplicate = True
                break
            j = j + 1

        if found_duplicate:
            already_added = False
            k = 0
            while k < len(duplicates):
                if duplicates[k] == numbers[i]:
                    already_added = True
                    break
                k = k + 1

            if not already_added:
                duplicates.append(numbers[i])

        i = i + 1

    return duplicates

print(find_duplicates([1, 2, 3, 2, 4, 1])) # [1, 2]

```

Example

Input: [1, 2, 3, 2, 4, 1] Output: [1, 2]

Complexity

Time: $O(n^2)$. Space: $O(d)$, where d is the number of distinct duplicates.

4. Two Sum Problem

Simple idea

Check every pair of numbers. Return the two indices when their sum equals the target.

How to solve it

12. Choose one number with the outer loop.
13. Check all numbers after it using the inner loop.
14. If their sum equals the target, return both indices.

Python code

```
def two_sum(numbers, target):
    i = 0

    while i < len(numbers):
        j = i + 1

        while j < len(numbers):
            if numbers[i] + numbers[j] == target:
                return [i, j]
            j = j + 1

        i = i + 1

    return [-1, -1]

print(two_sum([2, 7, 11, 15], 9)) # [0, 1]
```

Example

$2 + 7 = 9$, so the answer is indices 0 and 1.

Complexity

Time: $O(n^2)$. Space: $O(1)$.

5. Missing Number in an Array

Simple idea

Assume numbers should be from 0 to n and one number is absent. For each expected number, search the array. The value that is not found is missing.

How to solve it

15. Try every expected value from 0 to n .
16. Search the whole list for that value.
17. Return the first value that is not found.

Python code

```
def missing_number(numbers):
    expected = 0
    last_expected = len(numbers)

    while expected <= last_expected:
        found = False
        index = 0

        while index < len(numbers):
            if numbers[index] == expected:
                found = True
                break
            index = index + 1
```

```

        if not found:
            return expected

        expected = expected + 1

    return -1

```

```
print(missing_number([3, 0, 1])) # 2
```

Example

The expected values are 0, 1, 2, 3. Number 2 is absent.

Complexity

Time: $O(n^2)$. Space: $O(1)$.

6. Merge Two Sorted Arrays

Simple idea

Use one pointer for each sorted array. Add the smaller current value to the result, then move that pointer.

How to solve it

18. Start both pointers at index 0.
19. Compare the current values.
20. Add the smaller value and move its pointer.
21. After one array ends, copy the remaining values.

Python code

```

def merge_sorted_arrays(first, second):
    result = []
    i = 0
    j = 0

    while i < len(first) and j < len(second):
        if first[i] <= second[j]:
            result.append(first[i])
            i = i + 1
        else:
            result.append(second[j])
            j = j + 1

    while i < len(first):
        result.append(first[i])
        i = i + 1

    while j < len(second):
        result.append(second[j])
        j = j + 1

    return result

```

```

print(merge_sorted_arrays([1, 3, 5], [2, 4, 6]))
# [1, 2, 3, 4, 5, 6]

```

Example

The pointers compare 1 with 2, then 3 with 2, and continue in order.

Complexity

Time: $O(n + m)$. Space: $O(n + m)$.

7. Reverse a Linked List

Simple idea

Change every node so it points to the previous node instead of the next node.

How to solve it

22. Keep previous as None.
23. Save the current node's next node.
24. Point current.next to previous.
25. Move previous and current one step forward.

Python code

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def reverse_linked_list(head):
    previous = None
    current = head

    while current is not None:
        next_node = current.next
        current.next = previous
        previous = current
        current = next_node

    return previous

first = Node(1)
second = Node(2)
third = Node(3)
first.next = second
second.next = third

head = reverse_linked_list(first)
while head is not None:
    print(head.data, end=" ")
    head = head.next
# 3 2 1
```

Example

Before: 1 -> 2 -> 3. After: 3 -> 2 -> 1.

Complexity

Time: $O(n)$. Space: $O(1)$.

8. Sort an Array (Bubble Sort)

Simple idea

Bubble sort repeatedly compares neighboring values and swaps them when they are in the wrong order.

How to solve it

26. Repeat passes through the array.
27. Compare adjacent values.
28. Swap them when the left value is larger.
29. Stop early when no swap happens.

Python code

```
def bubble_sort(numbers):
    n = len(numbers)
    pass_number = 0

    while pass_number < n - 1:
        swapped = False
        index = 0

        while index < n - 1 - pass_number:
```

```

        if numbers[index] > numbers[index + 1]:
            temporary = numbers[index]
            numbers[index] = numbers[index + 1]
            numbers[index + 1] = temporary
            swapped = True
        index = index + 1

    if not swapped:
        break

    pass_number = pass_number + 1

    return numbers

```

```

print(bubble_sort([5, 2, 4, 1, 3]))
# [1, 2, 3, 4, 5]

```

Example

Large values move toward the end after every pass.

Complexity

Time: $O(n^2)$, best case $O(n)$. Space: $O(1)$.

9. FizzBuzz Problem

Simple idea

Print Fizz for multiples of 3, Buzz for multiples of 5, and FizzBuzz for multiples of both.

How to solve it

30. Start from 1.
31. Check divisibility by 15 first.
32. Then check 3 and 5 separately.
33. Otherwise print the number.

Python code

```

def fizz_buzz(limit):
    number = 1

    while number <= limit:
        if number % 15 == 0:
            print("FizzBuzz")
        elif number % 3 == 0:
            print("Fizz")
        elif number % 5 == 0:
            print("Buzz")
        else:
            print(number)

        number = number + 1

```

```
fizz_buzz(15)
```

Example

For 15, print FizzBuzz because 15 is divisible by both 3 and 5.

Complexity

Time: $O(n)$. Space: $O(1)$.

10. Remove Duplicate Numbers from a List

Simple idea

Build a new list. Add a number only when it does not already exist in the result.

How to solve it

34. Read one input value.
35. Search the result list for the same value.
36. Add it only when it is not found.

Python code

```
def remove_duplicates(numbers):
    result = []
    i = 0

    while i < len(numbers):
        exists = False
        j = 0

        while j < len(result):
            if result[j] == numbers[i]:
                exists = True
                break
            j = j + 1

        if not exists:
            result.append(numbers[i])

        i = i + 1

    return result

print(remove_duplicates([1, 2, 2, 3, 1, 4]))
# [1, 2, 3, 4]
```

Example

The first occurrence stays; later repeated occurrences are skipped.

Complexity

Time: $O(n^2)$. Space: $O(n)$.

11. Separate Vowels from a Word

Simple idea

Check every character manually. Put vowels in one string and all remaining characters in another string.

How to solve it

37. Read one character at a time.
38. Compare it with a, e, i, o, and u in both cases.
39. Add it to vowels or consonants.

Python code

```
def separate_vowels(word):
    vowels = ""
    others = ""
    index = 0

    while index < len(word):
        character = word[index]

        if (character == "a" or character == "e" or
            character == "i" or character == "o" or
            character == "u" or character == "A" or
            character == "E" or character == "I" or
            character == "O" or character == "U"):
            vowels = vowels + character
        else:
            others = others + character

        index = index + 1
```

```
    return vowels, others
```

```
vowels, others = separate_vowels("Shafiullah")
print("Vowels:", vowels)
print("Others:", others)
```

Example

For Shafiullah, vowels are a, i, u, a and the other letters are separated.

Complexity

Time: $O(n)$. Space: $O(n)$.

12. First Non-Repeating Character

Simple idea

For each character, count how many times it appears by scanning the whole string. Return the first character whose count is one.

How to solve it

40. Choose a character from left to right.
41. Count its occurrences with another loop.
42. Return it when the count equals one.

Python code

```
def first_non_repeating_character(text):
    i = 0

    while i < len(text):
        count = 0
        j = 0

        while j < len(text):
            if text[i] == text[j]:
                count = count + 1
                j = j + 1

        if count == 1:
            return text[i]

        i = i + 1

    return None

print(first_non_repeating_character("swiss")) # w
```

Example

In swiss, s and i repeat, but w appears only once and comes first.

Complexity

Time: $O(n^2)$. Space: $O(1)$.

13. Second Largest Number in a List

Simple idea

Track the largest and second-largest distinct values while reading the list once.

How to solve it

43. Initialize largest and second_largest as None.
44. If a value is greater than largest, move largest into second_largest.
45. Otherwise, update second_largest when the value is smaller than largest but larger than second_largest.
46. Ignore duplicate largest values.

Python code

```
def second_largest(numbers):
```

```

largest = None
second = None
index = 0

while index < len(numbers):
    value = numbers[index]

    if largest is None or value > largest:
        second = largest
        largest = value
    elif value != largest and (second is None or value > second):
        second = value

    index = index + 1

return second

```

```
print(second_largest([10, 5, 8, 10, 7])) # 8
```

Example

The largest distinct number is 10. The next largest distinct number is 8.

Complexity

Time: $O(n)$. Space: $O(1)$.

How to Explain a Coding Problem in an Interview

- Repeat the problem in your own words.
- Ask about input rules and edge cases.
- Explain the simple approach before coding.
- Write small, readable steps.
- Test the code with one normal example and one edge case.
- Mention time and space complexity.
- Explain possible improvements when your manual solution is not the fastest.

Important Edge Cases to Mention

- Empty string or empty list.
- Only one value.
- All values are duplicates.
- Negative numbers.
- No valid answer exists.
- Uppercase and lowercase letters.
- Second-largest value must usually be distinct.